Banking API Assessment — Technical Interview Guide

Candidate Interview Preparation Sheet • Node.js, Lambda, DynamoDB

37 TESTS PASSING | OPENAPI 3.0 CONFORMING | STRETCH GOALS COMPLETED

1. The 60-Second Elevator Pitch

WHAT TO SAY WHEN ASKED: "WALK ME THROUGH WHAT YOU BUILT"

*"I built a production-grade retail banking REST API following the **API Gateway** → **Lambda** → **DynamoDB** pattern, conforming strictly to the OpenAPI 3.0 specification.*

*I completed all **Core CRUD endpoints** for Customers and Accounts, plus all **Stretch Goal Transaction endpoints** (Deposits, Withdrawals, and atomic Transfers).*

My design focused on three engineering priorities:

- **Efficient Access Patterns:** Used a Global Secondary Index (GSI) with `Query` instead of expensive table `Scan`s for customer account listings.
- **Financial Consistency & ACID Transactions:** Implemented DynamoDB `TransactWriteItems` for cross-account transfers to guarantee all-or-nothing execution with zero risk of partial failure.
- **Rigorous Verification:** Built a 37-test automated test suite using `node:test` and `aws-sdk-client-mock` covering happy paths, input validation, and concurrency collisions with 0 failures."

2. Core Technical Questions & How to Answer

Q1 How did you design your DynamoDB access patterns, especially for listing customer accounts?

ANSWER SCRIPT

Point Reads (O(1)): Customers (`customerId`) and Accounts (`accountId`) use direct `GetCommand` by partition key.

Listing Accounts by Customer (`GET /customers/{id}/accounts`):

- In DynamoDB, the `Accounts` primary key is `accountId`. A naive approach would use a `Scan` with a filter expression. However, **Scan is an anti-pattern in production** because it scans the entire dataset ($O(N)$), consuming unbounded Read Capacity Units (RCUs) and increasing latency.
- Instead, I queried the `CustomerIdIndex` **Global Secondary Index (GSI)** using `QueryCommand` with `KeyConditionExpression: "customerId = :customerId"`. This targets only the relevant partition, achieving $O(1)$ partition lookup with minimal RCU consumption.
- I also verify that the customer exists beforehand so that invalid IDs return a proper `404 Not Found` rather than an ambiguous empty list.

Q2 **How did you implement transfers and handle partial failures or concurrency?**

ANSWER SCRIPT

The Partial Failure Risk: In financial transfers, if a debit succeeds but the subsequent credit fails (network timeout or system crash), customer funds vanish. If credit succeeds but debit fails, money is created.

My Solution — DynamoDB ACID Transactions (`TransactWriteItems`):

- I bundled the debit (`SET balance = balance - :amount`) and credit (`SET balance = balance + :amount`) into a single `TransactWriteCommand` . Both operations execute atomically across DynamoDB items—either both commit or both roll back.
- **Optimistic Concurrency Guards:** On the debit item, I added `ConditionExpression: "balance >= :amount AND #status = :openStatus AND currency = :currency"` .
- **Race Condition Handling:** If two transfers hit concurrently and deplete the balance, DynamoDB aborts the transaction with `TransactionCanceledException` . My handler catches this and returns a clean `409 Conflict` .
- **Production Idempotency:** In production, I would accept a `ClientRequestToken` / `Idempotency-Key` header to prevent duplicate execution upon client retries.

2. Core Technical Questions & How to Answer (Continued)

Q3 How did you handle deposits and withdrawals?

ANSWER SCRIPT

- **Atomic Balance Updates:** Rather than performing a vulnerable read-modify-write in Node.js, I used DynamoDB's native `UpdateCommand` with `SET balance = balance + :amount` (deposit) and `SET balance = balance - :amount` (withdraw).
- **Overdraft Prevention:** Withdrawals enforce `ConditionExpression: "balance >= :amount"`. This guarantees that balance can never drop below zero even under concurrent requests. If the condition fails, it returns `409 Conflict`.
- **Account State & Currency Validation:** Both operations verify that the account is `OPEN` and that the deposit/withdrawal currency strictly matches the account currency.

Q4 How did you handle input validation and financial precision?

ANSWER SCRIPT

- **Schema Conformance:** Matched all OpenAPI schemas. Malformed bodies, missing fields, or invalid types return `400 Bad Request` with `{ error: "BAD_REQUEST", message: "..." }`.
- **Floating-Point Precision:** IEEE-754 floating-point arithmetic can introduce fractional-cent artifacts (e.g. 0.1 + 0.2 = 0.30000000000000004). To prevent exploits and rounding issues, I enforced that all amounts must be finite numbers with at most **2 decimal places** (cents).
- **Status Rules:** Closed accounts are blocked from receiving funds or originating transactions. Self-transfers (same from and to account) are explicitly rejected with `400 Bad Request`.

Q5 How did you test your implementation?

ANSWER SCRIPT

- I implemented a dedicated unit test suite using Node's built-in `node:test` framework and `aws-sdk-client-mock`.
- Mocking the DynamoDB Document Client allows the entire suite of **37 tests across 11 suites** to execute in under 300ms without needing live Docker infrastructure.
- Tests verify happy paths (200/201), bad requests (400), not found (404), insufficient funds (409), GSI index query parameters, and transaction rollback handling.

3. Live Demo Quick Reference

Action	Command / URL	What to Point Out
1. Launch Server	`npm run dev`	1-command startup: auto-cleans ports, starts DynamoDB Local, seeds data, starts serverless.
2. Open Swagger UI	`http://localhost:3000/docs`	Show interactive OpenAPI documentation with all schemas and try-it-out controls.
3. Run Tests	`npm test`	Show 37 tests passing with 0 failures in terminal.
4. Point to Code	`src/handlers/`	Highlight clean file separation, GSI query in `accounts.js`, and `TransactWriteCommand` in `transactions.js`.

4. Technical Buzzword & Concept Summary

Key concepts to mention during the interview:

- **GSI vs. Scan:** Explain that GSI provides $O(1)$ partition isolation, whereas Scan is an $O(N)$ anti-pattern.
- **DynamoDB ACID Transactions:** Emphasize all-or-nothing atomicity via `TransactWriteItems`.
- **Optimistic Concurrency Control:** Use of `ConditionExpression` to prevent race conditions without table locks.
- **Idempotency:** Mention `ClientRequestToken` for safe client retries in production distributed systems.
- **Financial Precision:** Validation of 2-decimal place limits to prevent IEEE-754 precision issues.